IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts,Science,Commerce,and Management

Project – High Level Design

on

Natural Language Interface for Agriculture Infrastructure

Student Name: Aswanth M O

Industry Mentor:Mr. Shashank **GuidedBy:Mr. Praveen**

Table of Contents

1. Introduction

1.1. Scope of the document

This document explains the overall design of the Agriculture Chatbot project in simple terms. It shows the main parts of the system, how they work together, and how data moves from the user to the chatbot and back.

1.2. Intended Audience

This document is for:

- project reviewers
- faculty or guides
- developers
- testers
- anyone who wants a quick understanding of the system

1.3. System overview

The Agriculture Chatbot is a small application that helps users ask basic farming-related questions. It gives answers about:

- crop status
- soil condition
- fertilizer
- irrigation
- weather
- farming tips

- time

- command history

The project has two ways to use it:

A) website

b) command-line interface

Both use the same chatbot logic in the backend.

2. System Design

2.1. Application Design

The system is made of four main parts:

1. User Interface

   The user interacts through the website or CLI.

2. Web Server

   The Python server receives requests and sends responses.

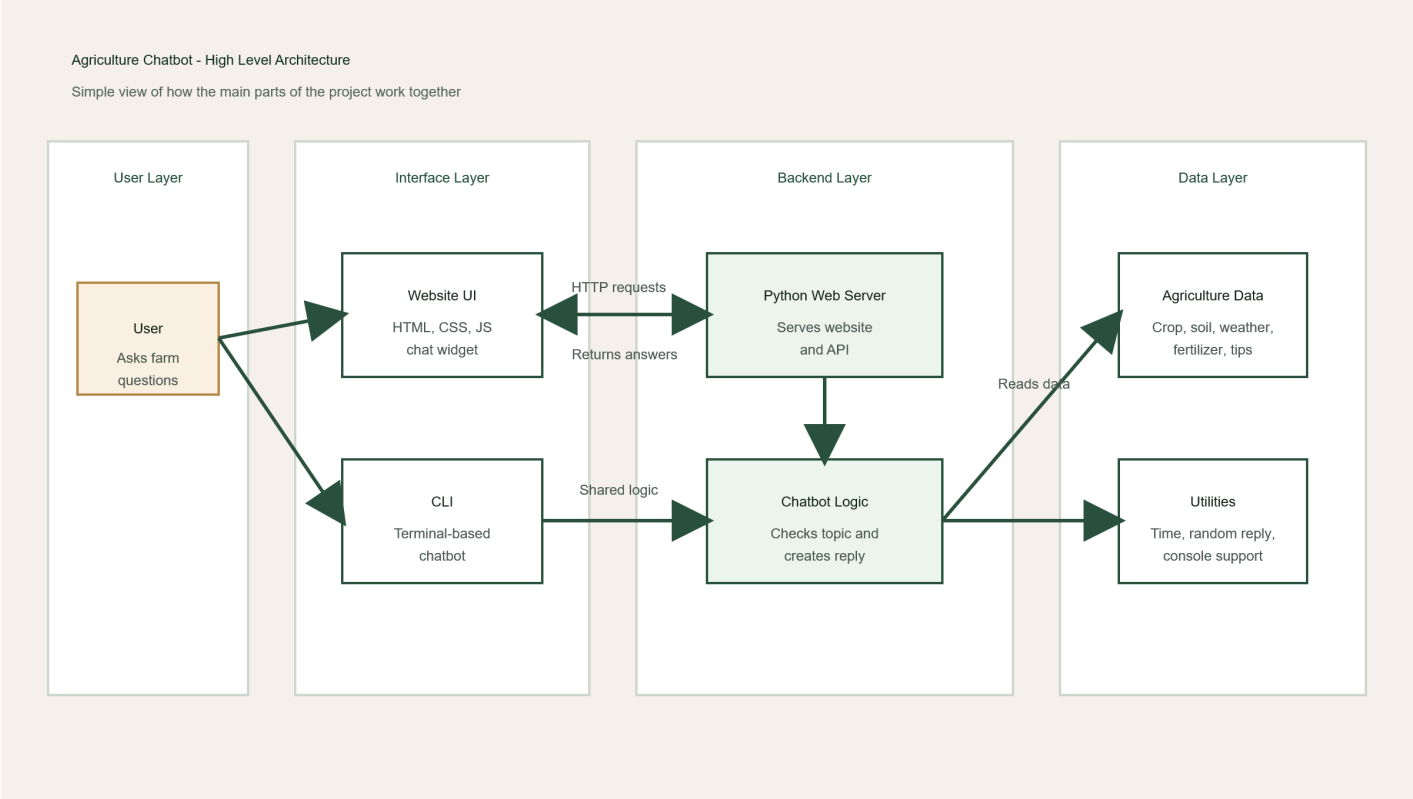
3. Chatbot Logic

   The command handler checks the user message and prepares the correct reply.

4. Data

   Agriculture information is stored in Python dictionaries and lists.

Simple Architecture Diagram



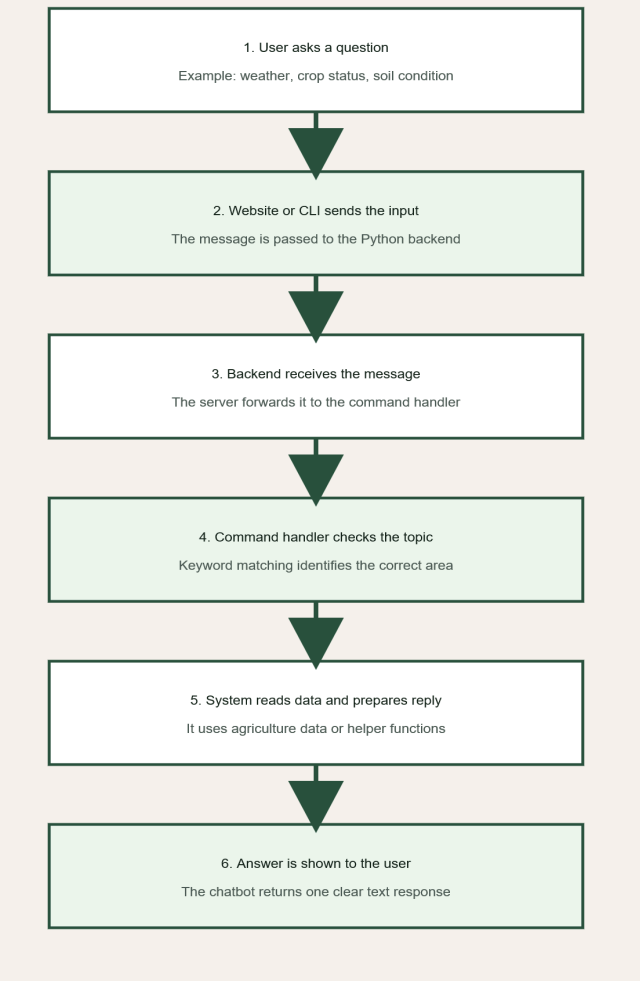
2.2. Process Flow

The system works in the following simple steps:

1. The user opens the website or CLI.
2. The user types a question.
3. The question goes to the Python backend.
4. The command handler checks the topic.

5. The chatbot picks the correct data or creates a small response.
6. The response is shown to the user.

Process Flow Diagram



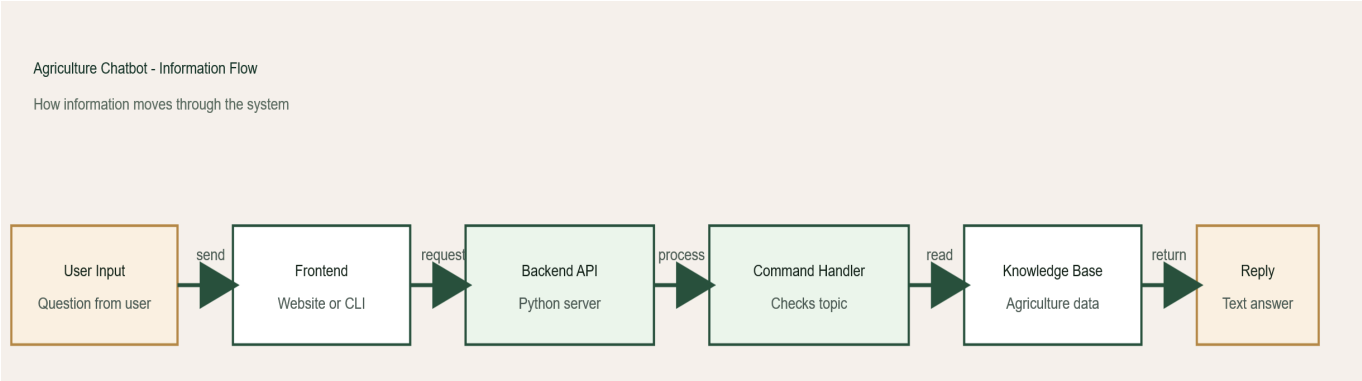
2.3. Information Flow

The information flow is simple:

- user sends message
- frontend sends message to backend
- backend checks the message

- backend reads stored agriculture data
- backend sends one text reply
- frontend displays the reply

Information Flow Diagram



2.4. Components Design

| Component | Purpose | File |

| --- | --- | --- |

| Website UI | Shows page and chatbot | `web/index.html`, `web/styles.css`, `web/app.js` |

| Web Server | Serves page and API | `web_app.py` |

| Chatbot Logic | Understands message and generates reply | `command_handler.py` |

| Data Store | Stores farming data in memory | `agriculture_data.py` |

| Utility Functions | Time, random messages, console support | `utils.py` |

| CLI | Terminal version of chatbot | `chatbot.py` |

2.5. Key Design Considerations

- The project is simple and easy to run.
- The same chatbot logic is reused for both web and CLI.
- No database is needed for the current version.
- The system is easy to extend with more topics later.
- The project is suitable for demo, academic, and small local use.

2.6. API Catalogue

| API | Method | Use |

| --- | --- | --- |

| `/` | `GET` | Opens the main website |

| `/styles.css` | `GET` | Loads website styling |

| `/app.js` | `GET` | Loads website behavior |

| `/api/welcome` | `GET` | Returns welcome text and quick actions |

| `/api/chat` | `POST` | Sends user message and gets bot reply |

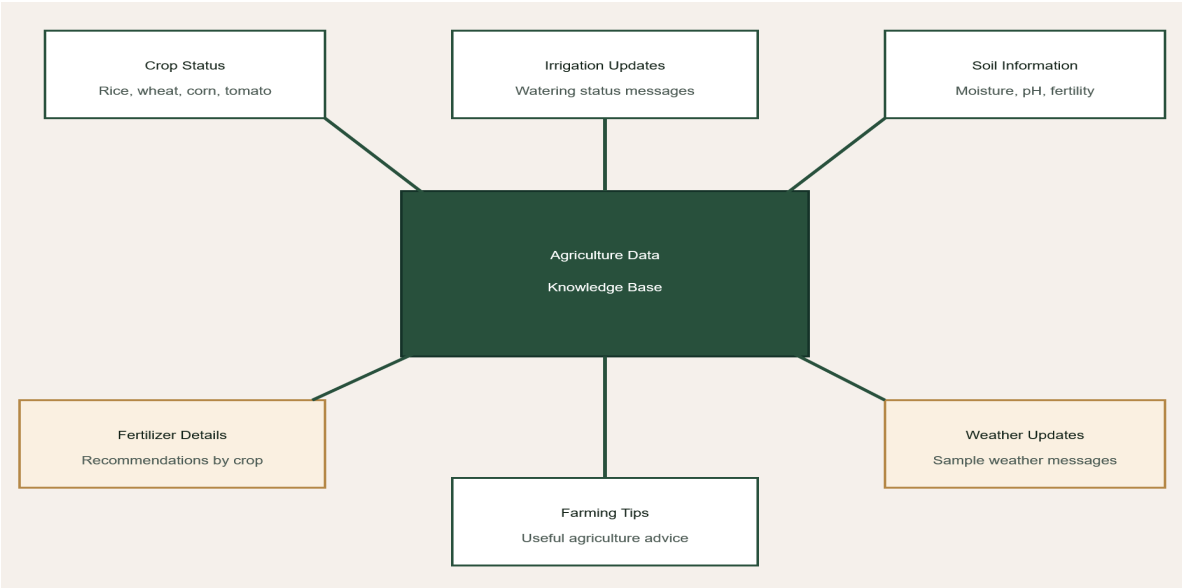
3. Data Design

3.1. Data Model

The project does not use a database. It uses simple Python data structures:

- dictionary for crop status
- dictionary for soil information
- dictionary for fertilizer details
- list for weather messages
- list for tips
- list for irrigation updates

Data Model Diagram



3.2. Data Access Mechanism

The backend directly imports data from `agriculture_data.py`.

There is no database connection and no external API call in the current system.

3.3. Data Retention Policies

- user chat is not saved permanently

- website history stays only until the page is refreshed

- CLI history stays only until the program closes

- no long-term storage is used

3.4. Data Migration

At present, data migration is not needed because the project uses static in-memory data. In future, the data can be moved to:

- JSON files

- CSV files

- a database

4. Interfaces

The project has two main interfaces:

- Web interface

  Users chat through a browser.

- CLI interface

  Users chat through the terminal.

The web interface talks to the backend using HTTP and JSON.

5. State and Session Management

The backend does not keep a full user session.

- the website stores chat history in browser memory
- the CLI stores chat history in a Python list
- the backend only uses the history sent by the frontend when needed

This keeps the system simple.

6. Caching

There is no special caching in the project.

- data is already available in memory
- static files may be cached by the browser
- no custom cache logic is used

7. Non-Functional Requirements

7.1. Security Aspects

- user input is treated as text
- chatbot replies are shown as text, not HTML
- the app runs locally by default
- no authentication is used in the current version

If the project is deployed publicly later, more security can be added.

7.2. Performance Aspects

- the system is fast because data is stored in memory
- the chatbot uses simple keyword matching
- the frontend is lightweight
- the server is enough for small-scale use

8. References

- `README.md`
- `web_app.py`
- `command_handler.py`
- `agriculture_data.py`
- `utils.py`

- `chatbot.py`
- `web/index.html`
- `web/styles.css`
- `web/app.js`